모델기반 RL과 몬테카를로 트리 탐색

Dyna-Q 재조명과 “공짜 모델”의 두 축 · MCTS 네 단계와 님 실습 · 멀티에이전트 동역학(선택) · World Models의 상상

Machine Learning 2 · 15주차

한국과학영재학교 (KSA)

Week 15

이번 주차 개요

이 주차를 마치면

1. 모델기반 RL을 known/learned $\times$ 학습/계획 두 축으로 정리
2. MCTS 네 단계를 님 게임에 구현, 횟수와 c 실험
3. **UCT** = 트리 위의 UCB, AlphaGo/AlphaZero 구조
4. (선택) 멀티에이전트의 **비정상성**과 보상 구조

- **15.1** Dyna-Q 재조명: known/learned $\times$ 학습/계획
- **15.2** MCTS: 네 단계 + UCT, 님 실습, AlphaZero
- **15.3** (선택) 멀티에이전트: 비정상성, 죄수의 딜레마
- **15.4** World Models: 학습된 모델 안에서 상상하며 계획하기

이 장의 주제 = "부분 계획": 전체 상태 공간을 계산하는 대신, 지금 필요한 갈림길만 깊게 계산한다.

15.1 모델기반 RL 개관: Dyna-Q 재조명과 “공짜 모델”

AlphaGo에서 MCTS로

- 2016년: 딥마인드 **AlphaGo**가 세계 최정상급 프로 이세돌을 꺾음
- 바둑판의 국면 수는 **우주의 원자 수보다 많다**고 알려져 있어서 "가치 근사"(DQN, Week 9)만으로는 부족
- AlphaGo의 핵심 무기 중 하나 = **몬테카를로 트리 탐색 (Monte Carlo Tree Search, MCTS)**
- Week 7.3에서 맛본 "모델을 활용한 계획"을, 명확한 규칙이 있는 게임에서 훨씬 강력하게 확장한 형태

이 장의 위치

지금까지의 ML2(DQN · PPO · 모방학습)는 대부분 **model-free**: 시행착오로 배움. 이번 장은 그 반대편 — **환경의 규칙이 주어졌을 때 그 모델을 이용해 미리 계획**하고 결정하는 방법. 이 두 축이 합쳐져야 AlphaGo/AlphaZero가 완성된다.

Dyna-Q 복습: 학습된 모델로 가상 경험 만들기

Week 7.3의 **Dyna-Q**는 세 가지를 **동시에** 했다:

1. 실제 환경과 상호작용해 **경험**을 쌓는다
2. 그 경험으로 간단한 모델 $\hat{P}(s'|s,a)$, $\hat{R}(s,a)$를 **학습**한다
3. 그 모델로 **가상의 추가 경험**을 만들어 Q값을 더 많이 갱신한다

모델기반 강화학습(Model-Based RL)

Dyna-Q 아이디어의 일반화 — 환경의 모델(전이확률 + 보상함수, 또는 그 근사)을 학습하거나 이미 알고 있다면, "실제로 행동해보지 않고도" 여러 수를 미리 시뮬레이션할 수 있다.

- Dyna-Q는 이 시뮬레이션을 **값 갱신**에 썼다
- 이번 장의 MCTS는 **지금 당장 어떤 행동을 할지 결정하는 데** 직접 쓴다 — 이 점이 차이

“모델을 안다”의 두 갈래

Dyna-Q의 모델 $\hat{P}, \hat{R}$은 **경험에서 쌓인** 것이었다 (각 (s, a)마다 “마지막으로 관찰된 (r, s')”을 기억하는 표).

known model(알려진 모델)

- 바둑·체스: 모델은 학습된 게 아니라 **규칙 자체** — 게임 규칙을 코드로 옮긴 것
- 데이터로부터 추정한 근사가 아니므로 **오차가 전혀 없는 완벽한 모델**

learned model(학습된 모델)

- Dyna-Q처럼 **데이터로부터 학습**한 모델
- 로봇 같은 실제 환경은 보통 이쪽 — 모델이 데이터로부터 배워야 하거나(또는 시뮬레이터로 만들거나) 아예 만들지 못한다

모델이 어디에서 오느냐(known/learned)와 모델을 무엇에 쓰느냐(학습/계획)는 두 개의 독립된 축이다.

두 축으로 모으기

- **학습**(값 개선)에 쓰면 — Dyna-Q의 전형: 모델로 만든 가상 경험을 Q 갱신에 쓸 뿐, "지금 이 수에서 무엇을 할지"를 직접 계산하지 않는다
- **계획**(결정)에 쓰면 — 15.2절 MCTS의 전형: (가상의) 행동 시나리오를 전개해 **지금 당장 둘 수**를 골라낸다

	known model (규칙이 주어짐)	**learned model** (데이터로 학습)
학습(값 개선)	가치반복·정책반복 (Week 4)	Dyna-Q (Week 7.3)
계획(지금의 결정)	MCTS·AlphaZero (15.2)	로봇의 **MPC** 등

- 바둑 = **위쪽줄 오른쪽칸**: 모델은 정확히 알지만 상태 공간이 너무 커서 "전체 계산"(Week 4)이 불가능 ⇒ "지금 필요한 것만 계산"(MCTS)으로 방향 전환
- **MPC(Model Predictive Control)**: 매 스텝마다 모델로 후보 행동 시퀀스를 미래로 시뮬레이션해, 결과를 가장 좋게 하는 시퀀스의 **첫 행동**만 취하는 제어 — 산업용 로봇에 널리 쓰이는 "모델로 계획하는" 제어기

역사적 맥락: 따로 자라온 두 계보

- MCTS의 계보는 학습 알고리즘 계보와 **따로** 자라왔다
- 2006년 Kocsis와 Szepesvari가 바둑 탐색에 제안한 것이 출발점, 그 선택 규칙 **UCT**는 **Week 2.2의 UCB 밴딧에서 직접 가져온 것** — "학습" 문헌의 도구를 "게임 AI"에 접목한 기술
- AlphaGo(2016)·AlphaZero(2017)가 여기에 **신경망을 엮어** 바둑·체스·시구를 석권한 뒤, 이 일대가 ML 공통 언어로 "모델기반 RL"이라는 이름 아래 묶이기 시작

"모델기반"은 우산 용어

고정된 '학파'가 아니다 — 기준은 **"모델을 이용해 실제 환경과의 상호작용을 줄이는가"**. 모델이 known/learned인지, 용도가 학습/계획인지는 그 안의 **독립된 축**.

14장의 시뮬레이터(MuJoCo·Isaac Sim)는 known model이 **주어진** 사례 — 13∼14장에서 "시뮬레이터 위에서 배우며 다닌"経験を "모델로 계획한다"는 관점에서 다시 짚는 장이다.

모델이 “공짜”인 경우: 체스와 바둑

체스·바둑 같은 게임에는 특별한 사정이 있다 —

- 규칙이 **완벽하게 알려져** 있다
- “이 수를 두면 판이 정확히 이렇게 바뀐다”는 전이확률 $P(s'|s,a)$를 Dyna-Q처럼 데이터로부터 배울 필요가 **전혀 없음** (Week 4 모델 기반 방법의 이상적 조건)
- 문제는 **상태 수가 너무 많다**: 바둑판의 가능한 국면 수는 약 2×10^{170}으로 추정

Week 4.3 가치반복 같은 “전체 계산”

모든 상태를 한 번씩 순회하며 계산 — 10^{170} 개 상태에서는 **근본적으로 불가능**.

MCTS(15.2)의 “지금 근처만 깊이”

“지금 당장 놓인 상황 근처만” 깊이 파고드는 탐색이 필요해진다 — 15.2절 MCTS가 풀려는 문제.

실습: 완전탐색이 가능한 경우(틱택토)

"모델을 안다"고 해서 항상 완전탐색(minimax)을 쓸 수 있는 건 아니다 — **상태 수가 관건**. 틀만 같고 상태 수가 훨씬 작은 틱택토(3×3)에서 검증:

```
WIN_LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
nodes_visited = 0

def winner(board):
    for a, b, c in WIN_LINES:
        if board[a] != 0 and board[a] == board[b] == board[c]:
            return board[a]
    return 0

def minimax(board, player):
    global nodes_visited
    nodes_visited += 1
    w = winner(board)
    if w != 0: return w
    if all(x != 0 for x in board): return 0   # 무승부
    scores = []
    for i in range(9):
        if board[i] == 0:
            board[i] = player
            scores.append(minimax(board, -player))
            board[i] = 0
```

결과: 549,946개 vs 10^{165}배 차이

```
완전탐색결과선공승
  (1=, 무승부0=, 후공승-1=): 0탐색한전체국면수
   : 549946
```

- 두 선수가 모두 최선을 다하면 틱택토는 **항상 무승부**로 끝난다
- 이 결론을 얻는 데 필요한 국면 수는 **549,946개** — 평범한 노트북에서 **1초도 안 걸려** 전부 훑을 수 있다
- 반면 바둑은 이보다 10^{165}배 이상 — 완전탐색이라는 "정직한 방법" 자체가 **아예 성립하지 않는 규모**

이 격차가 곧 MCTS의 이유

전체 트리를 다 보는 대신, **유망해 보이는 가지만 표본추출(sampling)로 골라 깊이 파고든다** — 15.2절의 MCTS.

손으로 계획 한 스텝: 알려진 모델에 가치반복

모델이 **완벽히 알려져** 있을 때 "계획"은 어떤 모습인가 — 가장 단순한 형태인 **가치반복**(Week 4.3)을 손으로 들여다본다.

- 3상태 예제: S(출발) $\to X$(중간) $\to T$(목표, 종료), 각 상태에서 행동 하나씩, 전이는 결정론적

$$S \xrightarrow{a} X \ (\text{보상 } 0), \qquad X \xrightarrow{a} T \ (\text{보상 } +1), \qquad \gamma = 0.9$$

참값 (거꾸로 대입)

$Q(T) = 0$, $Q(X) = 1 + 0.9 \cdot 0 = 1$,
$Q(S) = 0 + 0.9 \cdot 1 = 0.9$ — 손에 바로 나온다.

가치반복 한 반복

모든 (s, a)에 대해 $Q(s,a) \leftarrow r + \gamma \max_{a'} Q(s', a')$.
주의: **동기화(synchronous) 갱신** — 새 값을 **오래된 Q**로 계산한 뒤 한꺼번에 바꿔 넣는다.

가치반복의 반복 1→3: 값이 "한 칸씩" 퍼져 나간다

반복	계산	결과
1 (초기 Q 전부 0)	$Q(X) \leftarrow 1 + 0.9 \cdot Q(T)$, $Q(S) \leftarrow 0 + 0.9 \cdot Q(X)$	$Q(X) = 1$, $Q(S) = 0$ (아직 $Q(X)$가
2	$Q(S) \leftarrow 0 + 0.9 \cdot Q(X) = 0.9$	$Q(S) = 0.9$, $Q(X)$는 변하지 않음
3	아무 값도 변하지 않음	**수렴**

- $Q(S)$가 0에서 0.9로 도약하는 데 **정확히 2번**의 반복이 걸렸다
- 반복 1: 목표에서 **1칸** 떨어진 X만 "+1이 2칸 뒤에 있다"는 사실을 알게 됐고, 2칸 떨어진 S는 아직 0
- 반복 2: 값이 엣지 $S \rightarrow X$를 타고 **한 칸 뒤로 퍼져** S까지 도착

본질: 한 반복당 한 칸

값 정보가 모델 그래프를 따라 **정확히 한 상태만큼** 퍼져 나간다 — 목표에서 d칸 떨어진 상태는 정확히 d번 반복, L개 링크 체인의 전체 수렴은 $L+1$**번째 반복**. (7.3절 5×5 격자: 최단 경로 8스텝 ⇒ 9번 반복에 수렴.)

깊이 시뮬레이션 하나가 가치를 "압축"한다

"한 번에 한 칸"을 다른 관점에서 보자 — 우리는 **모델을 안다**(규칙을 안다)고 하니, 체인을 **한 번에 전부 시뮬레이션**할 수 있다:

$$S \xrightarrow{a} X \xrightarrow{a} T$$

- 두 스텝만 시뮬레이션하면 보상 $+1$을 **바로** 본다
- 가치반복이 2번 반복에 걸쳐 발견한 것과 **동일한 정보를, 깊이 있는 시뮬레이션 하나가 압축**해 준다

MCTS와 이 대응

MCTS의 "시뮬레이션(끝까지 전개) + 역전파(결과를 지나온 노드로 전달)"는 정확히 이 "깊이 시뮬레이션 + 결과 뒤로 전파"의 **표본추출 버전** — 전체 체인이 아니라 "유망해 보이는" 체인을, 전체 상태를 대신 샘플링으로 골라 처리한다는 것만 다를 뿐. 15.2절에서 이 대응을 단계별로 다시 펼쳐 본다.

실습: Q-learning vs Dyna-Q — 가속을 숫자로

위 손계산은 모델이 **완벽히 알려진** 경우. 일반적인 경우는 모델을 **배워야 하는** 경우(Dyna-Q) — 학습된 모델로 계획하는 것이 학습을 실제로 얼마나 가속하는지, 7.3절의 5×5 격자로 재본다.

- 환경: 출발 $(4,0)$, 목표 $(0,4)$, 4방향 이동, 이동마다 보상 -1, 목표 도착 시 0으로 종료, $\gamma = 1$, $\alpha = 0.1$, $\varepsilon = 0.1$
- 참값(손계산): 최단 경로 8스텝 $\Rightarrow$ 최적 리턴 -7 — $Q^*(\text{출발}, \text{위}) = Q^*(\text{출발}, \text{오른쪽}) = -7$, "아래/왼쪽"은 -8
- known model이면 가치반복 9번 반복(계산 900회)으로 끝남 — **계획이 사실상 공짜**인 경우
- learned model(Dyna-Q): 실제 스텝 1회당 모델 엔트리 1개만 늘어나므로 모델 커버리지가 **경험과 함께 서서히 성장**

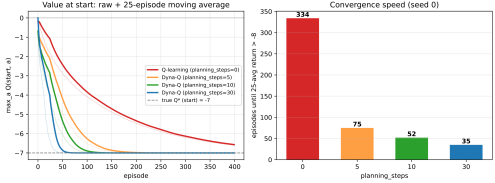
$planning_steps \in \{0, 5, 10, 30\}$을 400 에피소드로 비교 (시드 0, 시드 1∼3도 유사한 범위).

결과: 334 에피소드 → 35 에피소드

설정	“거의 최적” 최초 진입†	400에피소드 시 max Q(출발)
가치반복 (known model)	9번 반복	정확히 -7.00
Q-learning ($planning_steps$=0)	334 에피소드	-6.61
Dyna-Q ($planning_steps$=5)	75	-7.00
Dyna-Q ($planning_steps$=10)	52	-7.00
Dyna-Q ($planning_steps$=30)	35	-7.00

† 25에피소드 평균 리턴 > -8

- **(1) 가속은 극적**: 실제 상호작용 35회로, 계획 없이는 334회가 걸렸던 학습이 끝남(약 **9.5배**). “실제 스텝 1회 = 로봇 이동 1초” 환경이면 쓸 수 있는 수준까지의 **벽시계**가 같은 비율로 줄어듦
- **(2) 한계 효과 감소**: 0→10은 334→52(6.4배) 크게 빨라지나, 10→30은 52→35(1.5배)로만 — 계획 계산량 2배가 수렴 2배가 아님
- $planning_steps$도 α, ε처럼 **검증 성능으로 튜닝하는 하이퍼파라미터**



위: max Q(출발) 학습 곡선 — 0(빨강)이 에피소드 330쯤 -7 도달, 10/30(초록/파랑)은 40∼50. 회색 점선 = 참값 -7. 아래: “거의 최적” 최초 진입 에피소드 수.

루프의 대응: Dyna-Q와 AlphaZero는 같은 구조

마지막 점이 이 장의 줄거리를 잇는다:

두 루프는 동일한 구조

Dyna-Q 루프(learned model ↔ 계획으로 값 개선)와 **AlphaZero 루프**(신경망 ↔ MCTS 탐색)은 **동일한 구조** — "모델" 자리에 학습된 신경망, "Q 테이블" 자리에 가치 헤드, "계획 갱신" 자리에 MCTS 탐색이 앉을 뿐.

- 7.3절의 루프 = "모델로 Q를 개선하는 **학습**"
- 15.2절의 MCTS = "모델로 지금의 결정을 계산하는 **계획**"
- 즉, **같은 두 요소 루프를 학습/계획이라는 두 축에서 다시 보는** 셈

모델이 known인지 learned인지, 학습에 쓰는지 지금의 결정에 쓰는지 — 두 축의 교차점에 무엇이 앉는가?

자주 하는 실수: “모델이 있으니 학습이 필요 없다”

모델기반 RL을 처음 배울 때의 흔한 오해 —

“환경의 규칙을 완벽히 안다면, Week 4의 가치반복 같은 planning으로 바로 최적 정책을 계산할 수 있으니 **신경망 학습이 아예 필요 없다**”

- 이건 **상태 수가 작을 때만** 맞는 말
- 바둑처럼 상태 수가 천문학적이면, “모델을 안다”는 것과 “그 모델로 최적 행동을 **실제로 계산**할 수 있다”는 것은 **전혀 다른 얘기** — 그래서 AlphaGo/AlphaZero는 여전히 신경망을 **학습**시킨다
- 반대로 Dyna-Q처럼 모델이 **학습된** 것일 때는 **모델 오차**가 Q값에 누적: 초기에 몇 안 되는 경험으로 학습한 부정확한 모델로 가상 경험을 너무 많이 만들면, 그 부정확함이 Q값에 스며들어 오히려 **학습을 방해**

기억할 것

“모델기반”이 공짜로 정확한 답을 주는 게 아니다 — **모델의 정확도(또는 완전성)**과 **상태 공간의 크기 둘 다**가 계획의 실현 가능성을 결정한다.

역방향 실수: 모델이 known하면 학습 불필요?

"모델을 완벽히 안다면(known model), 학습은 전혀 필요 없다"는 역방향 버전 —

- 틱택토 실습: 이 주장은 **상태 공간이 작을 때만** 성립
- 바둑: 상태 공간이 너무 커서 **완벽한 모델이 있어도** 전체 공간으로 최적 행동을 계산할 수 없다

AlphaGo/AlphaZero가 규칙을 완전히 알면서도 신경망을 학습시킨 이유는 "모델을 배우기 위해"가 아니라 "탐색 공간을 압축하기 위해서".

모델(규칙)이 말하는 것

"이 수를 두면 판이 어떻게 변하는지" — 답이 **어디에 있는지**.

신경망이 말하는 것

"어떤 갈림길부터 들여다볼 가치가 있는가" — **어디를 볼지**.

- MCTS는 신경망의 힌트를 따라 유망한 소수 가지만 골라 깊이 시뮬레이션 — 제한된 시간(예: 한 수당 2초)에 수백∼수천 번의 시뮬레이션으로 "모델이 아는 답"을 채굴
- 모델을 몰랐다면 시뮬레이션 한 번도 못 돌렸을 텐데, 모델이 있어도 **탐색 방향**을 몰랐다면 시뮬레이션을 엉뚱한 가지만에 흘릴 뻔

"모델을 안다"는 것은 계획의 **재료**를, "학습"은 계획의 **방향**을 준비해 준다 — 둘은 서로를 대체하지 않는다.

자주 묻는 질문(15.1)

- **Q: Week 4 가치반복도 "모델기반"인데, 왜 이 장에서 다시 이 용어?** — 차이는 **스케일과 쓰임새**: Week 4는 상태가 적어 정책 **전체**를 미리 계산, 이번 장(Dyna-Q, MCTS)은 상태가 너무 많아 "지금 이 상태에서 무엇을 할지" **만** 계산
- **Q: 모델기반이 model-free보다 항상 좋은가?** — **아니** — 정확한(또는 잘 학습된) 모델이 있을 때 데이터 효율이 훨씬 좋지만, 모델 학습 비용과 **오차 누적** 위험이 있음. 로봇 팔처럼 물리 법칙이 복잡한 환경은 Week 9–13의 model-free 쪽이 더 실용적인 경우 많음
- **Q: known model이면 learned model 얘기를 안 해도 되나?** — 실제 환경 대부분이 바둑처럼 규칙이 완전히 알려진 게임이 **아님**(로봇·화학 공정·교통·인간-기계 상호작용). "공짜 모델"은 예외적 특권, **일반은 learned model** — 2010년대 이후 연구(world model 계열)의 주목적도 이 케이스 (15.4절)

자주 묻는 질문: DQN 경험 재사용과의 관계

Q: known model 경우와 Week 9 DQN의 경험 재사용 (experience replay)은 관계가 있나?

둘 다 "**손에 있는 정보를 더 많이 쓰자**"는 기술이지만 **재사용의 원천이 다르다**:

경험 재생(experience replay)

- **실제** (s, a, r, s') 튜플을 버퍼에서 다시 뽑아 쓰기 때문에 **모델 오차가 없음**
- 단, **경험하지 않은 전이는 만들 수 없다**
- "**안전하지만** 새것을 못 만든다"

모델 계획(Dyna)

- 모델로 전이를 **생성** — 경험이 커버하지 않은 상태까지 **갈 수 있음**
- 단, **모델 오차를 짊어진다**
- "새것을 만들 수 있지만 **오차가 동반**"

9장 이후의 모델기반 DQN 변형들은 학습된 신경망 모델을 얹어 이 둘을 **결합**하는 방향으로 나아간다.

확인 문제(15.1)

1. **체인 전파**: 체인을 $S \to X_1 \to \cdots \to X_8 \to T$(10개 상태, 링크마다 보상 0, 마지막 링크만 $+1$, $\gamma = 0.9$)로 늘리면, 가치반복에서 $Q(S)$가 0이 아닌 값이 되려면 몇 번 반복이 필요한가? known model로 시뮬레이션만 하면 몇 스텝이면 충분한가? (힌트: “한 번에 한 칸” vs “한 스텝에 한 칸”.)
2. **틀린 모델**: `dyna_q_grid`의 모델 갱신 줄 `model[(s, a)] = (r, ns)`를 `model[(s, a)] = (r, (4 - ns[0], 4 - ns[1]))`로 (다음 상태를 판 대각 반대 자리라 예측하는) 틀린 모델로 바꿔 $planning_steps{=}30$, 400 에피소드 학습 후 출발점 $(4, 0)$의 max Q와 25에피소드 평균 리턴을 기록. “아래/왼쪽” 엔트리가 다음 상태를 **목표** $(0, 4)$로 예측하는 것을 확인하고, 이 엔트리가 Q값과 greedy 정책을 어디로 끌어당기는지 설명.
3. **도로 vs 목적지**: $planning_steps{=}0$(순수 Q-learning)도 400 에피소드 후 greedy 정책이 25개 상태 전 Q^*와 일치(불일치 0/25). 그럼에도 계획이 실제로 “가속”한 것은 무엇인가? (“처음 진입” 열 참고.)
4. **루프 대응**: Dyna-Q 루프의 세 요소(학습된 모델, Q 테이블, 계획에 의한 가상 갱신)를 AlphaZero의 구성 요소(신경망 정책/가치 헤드, MCTS)와 짝짓고 각 대응을 한 문장으로.

15.1 핵심 정리

모델기반 RL = 모델을 이용해 실제 환경과의 상호작용을 줄이는 방법들의 묶음

모델의 **출처**(known/learned)와 **용도**(값 개선 = 학습 / 지금의 결정 = 계획)이라는 **두 축이 독립적**.

방법	두 축에서의 위치
가치반복	known + 학습
Dyna-Q (Week 7.3)	learned + 학습
MCTS	known + 계획
AlphaZero	신경망이 모델의 역할을 맡은 확대판 Dyna 루프

- "모델을 안다"는 것이 계획의 **재료**를, "학습"은 계획의 **방향**을 준비해 준다 — **둘은 서로를 대체하지 않는다**

모델기반 RL은 하나의 알고리즘이 아니라 "모델이 실제 상호작용의 일부를 대신할 수 있다"는 한 가지 아이디어로 묶인 **방법들의 가족**. 다음 15.2절: 두 축의 가장 날카로운 교차점 — known model로 "지금의 결정"을 계산하는 **MCTS**.

15.2 몬테카를로 트리 탐색: 이론과 실습

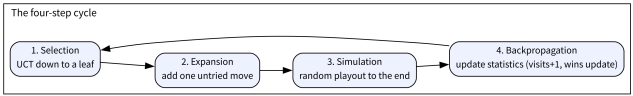
MCTS: 네 단계의 반복

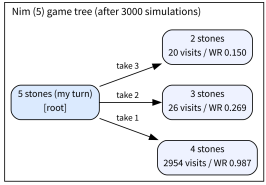
MCTS는 현재 상태에서 시작해, **정해진 횟수**만큼 다음 네 단계를 반복해 "어떤 수가 가장 유망한가"에 대한 **통계**를 쌓는다:

1. **선택(Selection)**: 루트(현재 상태)에서, "지금까지의 성과" + "아직 덜 시도해본 정도"를 함께 고려하는 규칙 (Week 2.2의 UCB와 **정확히 같은 발상**)으로 자식을 골라 내려가, 아직 다 확장되지 않은 노드까지
2. **확장(Expansion)**: 그 노드에 새로운 자식(아직 안 가본 수)을 **하나** 추가
3. **시뮬레이션(Simulation/Rollout)**: 그 새 노드에서 (간단한 정책으로, 극단적으로 **완전 무작위**로) **끝까지** 플레이 — Week 5의 몬테카를로처럼 결과 (승/패)를 얻는다
4. **역전파(Backpropagation)**: 그 결과를 지나온 모든 노드에 통계로 반영(방문 +1, 승률 갱신)

이 "역전파"는 ML1 Week 9의 역전파와 **이름만 같을 뿐 완전히 다른 개념**: 신경망의 그래디언트가 아니라, 트리의 **승패 통계**를 전달한다.

네 단계가 순환하는 구조





- 왼쪽: 님 게임(돌 5개) 게임 트리 — 루트에서 “1개/2개/3개 가져가기” 세 갈림
- 각 자식 노드에 **방문 횟수 + 승률**이 쌓임 (3000회 시뮬레이션 후 실제 값: **2954/0.987**, 26/0.269, 20/0.150)
- 오른쪽: 선택은 트리를 **내려가고**, 역전파는 지나온 **모든** 노드의 통계를 갱신한 뒤 다음 선택으로 돌아온다

- 수백~수만 번 반복한 뒤, 루트에서 **가장 많이 방문된** 자식(반복적으로 가장 유망하다고 확인된 수)을 실제로 둘 수로 선택
- “가장 많이 방문된 것”과 “승률이 가장 높은 것”은 **대개 일치하지만 항상 같지는 않다** (FAQ에서 다룬다)

계보: moa*에서 UCT까지

- “**무작위로 끝까지 해보고**(시뮬레이션), 그 결과를 트리의 통계로 적재적재하게 나누어 모은다(선택 + 역전파)”는 이 아이디어의 계보는 1990년대 **오툴로 (Othello)** 프로그램의 **moa*** 알고리즘까지 거슬러 올라감

UCT (Upper Confidence bound applied to Trees)

2006년 Kocsis와 Szepesvari가 이 가문의 방법들을 **UCT** 라는 이름으로 정리하며, Week 2.2의 **팔레트 아머드 밴딧** 문제를 트리 탐색에 연결하는 **정확한 수식**을 줌 — 이 UCT가 오늘 배울 선택 규칙.

- 이후 MCTS는 바둑·체스·광물자원 게임 등 “경우의 수가 너무 많아 **완전탐색이 불가능한**” 문제들의 **표준 계획 알고리즘**이 됨
- 2016년 AlphaGo의 핵심 구성요소로

하나의 시뮬레이션을 따라가보자

루트(돌 5개, **내 차례**)에서 시작:

1. **선택** — 루트에 자식이 **하나도 없으므로** 루트에서 즉시 정지
2. **확장** — 아직 안 가본 수(예: “2개 가져가기”)를 무작위로 하나 골라 자식으로 추가
3. **시뮬레이션** — 새 노드(돌 3개, **상대 차례**)부터 무작위 플레이로 게임 끝까지 두어 승자를 판정
4. **역전파** — 루트와 그 새 자식, **두 노드 모두**에 “방문 1회”를 더하고, 승자가 “그 노드로 **두어 도달한 플레이어**”라면 승 1회도 더함

“모든 수를 최소 한 번” 보장은 여기서 온다

첫 3회 시뮬레이션은 세 갈림의 각각을 한 번씩 “열어보는” 데 쓰임 — **방문 0인 자식은 항상 우선**되기 때문. 4회차부터 선택 단계의 UCT가 실제로 **작용**하기 시작한다.

UCT: 선택 단계의 규칙

"성과 + 덜 시도해본 정도"는 UCB1의 수식과 **정확히 같은 형태**. 노드 s에서 자식 a를 고를 때:

$$\mathrm{UCT}(s,a) \;=\; \frac{Q(s,a)}{N(s,a)} \;+\; c\sqrt{\frac{\ln N(s)}{N(s,a)}}$$

- $Q(s,a)$ = 자식 a의 누적 승(수), $N(s,a)$ = 방문 횟수, $N(s)$ = 부모 s의 방문 횟수
- 첫 항 = **활용**("이 수의 승률이 얼마나 높은가"), 두 번째 항 = **탐험**("이 수를 얼마나 덜 보았는가")

탐험 항의 감소

탐험 항의 크기는 대략 $1/\sqrt{N(s,a)}$로 감소 — 한 수가 **4번** 시도되면 탐험 보너스가 **절반**, **100번**이면 **1/10**으로 줄어듦. 그래서 "명백히 낮은 수"는 소수 몇 번 추가 시도 후 **방치**되고, "경합 중인 수들"은 비례해서 계속 분배된다.

UCT 선택 — 코드로

```python
import math

def uct_select(node_children, parent_visits, c=1.4):
    # node_children: {move: (wins, visits)}
    # 과UCB1 정확히같은형태   -- Week 를2.2 그대로게임트리에적용한것
    return max(node_children, key=lambda m: node_children[m][0] / node_children[m][1] +
               c * math.sqrt(math.log(parent_visits) / node_children[m][1]))
```

숫자로 체감 — 부모 **16회** 방문, 자식 세 개 (A: 승 7/방문 10, B: 승 3/방문 5, C: 승 0/방문 1), $c = 1.4$일 때:

- A: $7/10 + 1.4\sqrt{\ln 16/10} = 0.70 + 0.27 = \mathbf{1.437}$
- B: $3/5 + 1.4\sqrt{\ln 16/5} = 0.60 + 0.59 = \mathbf{1.643}$
- C: $0/1 + 1.4\sqrt{\ln 16/1} = 0.00 + 1.44 \approx \mathbf{2.331}$

승률 0인 C가 선택된다.

왜 승률 0인 C가 선택되는가

"**지금까지 한번도 이겨본 적 없다**"는 수인데 왜 C인가:

- 아직 **1번**밖에 시도하지 않았으므로, **"진짜 나쁜 수"**일 가능성과 **"아직 제대로 평가되지 못한 수"**일 가능성을 **분간**할 수 없음

C를 몇 번 더 보내면 —

- **진짜 나쁘다면**: 승률이 확인되고 탐험 항이 줄어 **자연스럽게 방치**
- **운 좋게 괜찮다면**: 활용 항이 올라와 본격적 **승부수**가 됨

Week 2.2의 트레이드오프

탐험-활용 트레이드오프가 **트리 탐색에서 그대로 작동**하는 모습. (15.3절 연습문제 1이 바로 이 계산을 빈칸으로 준다.)

MCTS 전체: 게임 규칙 + 노드 (님)

게임 = **님(Nim)**: 돌을 1∼3개씩 번갈아 가져가고 **마지막 돌을 가져가는 쪽이 이김**. 노드는 **그 노드에서 "이동한 플레이어"(mover)의 관점**의 통계를 담는다는 점에 주목 — **역전파가 무엇을 어떻게 더하는지**가 전부 여기서 결정된다:

```
import math
import random

def nim_moves(state):
    """이 상태에서가능한수  : 남은돌수만큼  , 개개개1/2/3 가져가기."""
    return list(range(1, min(3, state) + 1))

def nim_step(state, move, player):
    """개를move 가져간다: 남은돌이줄고  , 상대차례가된다  ."""
    return max(0, state - move), -player

def nim_terminal(state):
    return state == 0

def nim_rollout(state, player, rng):
    """시뮬레이션롤아웃(): 완전히무작위정책으로끝까지  . 승자마지막
    ( 돌을가져간플레이어  ) 반환. 이미종료된상태면막이동한쪽이승자
       ."""
    if nim_terminal(state):
```

MCTS 전체: 4단계 조립

```
def mcts_nim(stones, iters, c=1.4, seed=0):
    """돌이 개인stones 님게임에서루트위치의    MCTS. 루트노드반환  ."""
    rng = random.Random(seed)
    root = MCTSNode(stones, player=1, parent=None)
    for _ in range(iters):
        # 1) 선택: 아직" 다확장되지않은리프까지   " 로UCT 내려간다.
        node = root
        while not nim_terminal(node.state) and len(node.children) == len(nim_moves(node.state)):
            node = max(node.children.values(), key=lambda ch:
                       ch.wins / ch.visits + c * math.sqrt(math.log(node.visits) / ch.visits)
                       if ch.visits else float('inf'))
        # 2) 확장: 안가본수하나를자식으로추가     . 이미( 끝난상태면확장생략   )
        if not nim_terminal(node.state):
            unexpanded = [m for m in nim_moves(node.state) if m not in node.children]
            m = rng.choice(unexpanded)
            next_state, next_player = nim_step(node.state, m, node.player)
            node.children[m] = MCTSNode(next_state, next_player, parent=node)
            leaf = node.children[m]
        else:
            leaf = node
        # 3) 시뮬레이션: 리프에서무작위정책으로끝까지   , 승자판정
        winner = nim_rollout(leaf.state, leaf.player, rng)
        # 4) 역전파: 지나온모든노드방문    +1; 승자가면 mover 승 +1
        cur = leaf
```

세 자잘한(그러나 치명적인) 지점 ①: 선택의 정지 조건

- "**아직 다 확장되지 않은 리프**까지" — `while` 조건이 `len(node.children) == len(nim_moves(state))`인 이유: 자식이 **일부만** 있는(부분 확장) 노드에서 UCT로 계속 내려가면 안 된다
- 아직 한 번도 열어보지 않은 자식이 남아 있으면, **우선 그걸 열어야**(확장의 "모든 수를 최소 한 번" 보장) 트리의 통계가 **공정**해진다

이 조건을 `while node.children:`으로 잘못 쓰면

모든 방문이 **첫 번째로 열린 자식 하나**에 집중되는 버그가 나온다 — 아래 "버그 투어"에서 실증.

세 지점 ②: 역전파의 관점 “mover”에게 승을

- **mover** = “이 노드로 **이동한** 플레이어”
- 자식 노드라면 **부모의 차례**인 플레이어 (`cur.parent.player`), 루트라면 이동한 플레이어가 없으므로(`parent is None`) 현재 차례 플레이어($cur.player$)를 **대신 쓰**되 — 루트 자체의 통계는 자식 선택에 쓰이지 않으므로 ($\ln N(s)$에 $N(s)$만 들어가) **무해**

이 한 줄이 틀리면 — “차례인 플레이어”에게 승을 더하면

UCT가 **상대에게 유리한 수를 좋은 수로 착각**하게 된다. 왜 그런지는 버그 투어에서 **숫자로** 본다.

세 지점 ③: 터미널 상태의 확장

- 게임이 이미 끝난 노드(돌 0개)에서는 `nim_moves`가 **빈 리스트**를 돌려주므로, 터미널 검사를 하지 않고 확장을 시도하면 `IndexError`
- `nim_rollout`도 **같은 이유**로 첫 줄에 터미널 처리가 있다

"마지막 돌을 가져간 쪽이 이긴다"를 코드로

"누가 마지막에 이동했는가"를 `-player`(차례가 **전달되기 전**의 플레이어)로 판정해야 한다는 점에 주의 — `nim_rollout`이 종료 상태에서 `-player`를 반환하는 이유.

왜 추정값이 수렴하는가: 무작위 기준선

기준선 먼저 — MCTS 없이, "1개 가져가기"를 둔 뒤 남은 게임 (돌 4개, 상대 차례)을 **양쪽 다 무작위**로 끝까지 할 때 나의 승률은? "돌 n개에서 **차례인 쪽**의 무작위 승률"을 $P(n)$이라 하면, 규칙 그대로 재귀된다:

$$P(1) = 1, \qquad P(n) = \frac{1}{k(n)} \sum\nolimits_{m} \big(1 - P(n-m)\big)$$

$k(n)$ = 가능한 수의 개수, $n \le 3$이면 $k = n$.

- 손으로: $P(1) = 1$, $P(2) = (1 - P(1) + 1)/2 = 1/2$,
  $P(3) = (1 - P(2) + 1 - P(1) + 1)/3 = 1/2$,
  $P(4) = (1 - P(3) + 1 - P(2) + 1 - P(1))/3 = 1/3$
- 5개에서 "1개 가져가기"(상대에게 4개 남기는 수)의 무작위 승률 $= 1 - P(4) =$ **2/3** $\approx$ **67%**, "2개/3개 가져가기"는 각각 $1/2$ (실제 측정: 0.671 / 0.505 / 0.496)

순수 무작위 플레이로는 승패를 약 17%p 차이로밖에 구분할 수 없다 — 그런데 MCTS의 추정 승률은 98.7%, 67%를 훨씬 뛰어넘는다. 이 갭이 MCTS의 핵심.

트리 + 롤아웃의 "합성": 기억과 표본추출의 분업

MCTS의 승률 추정치는 "**트리가 기억한 것**"과 "**롤아웃이 표본추출한 것**"의 **합성**.

- "1개 가져가기" 자식으로 내려간 뒤, 상대가 돌 4개에서 아무 수를 둬도 **나에겐 필승 답이 존재**(상대가 m개를 가져가면 내가 $4-m$개를 가져가면 됨)
- 트리가 이 **"2수짜리 구조"**를 노드의 승패 통계로 저장하면, 그 아래로 나가는 모든 롤아웃은 사실상 "이미 결정된 승부"를 **재확인**하는 데 불과 — 승률 추정이 **100%에 수렴**

분업

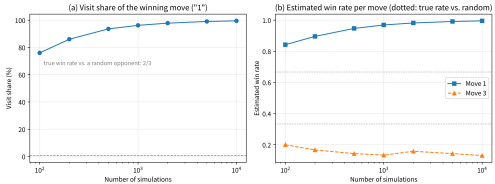
트리 = "기억할 수 있는 부분"(짧은 범위의 강제 구조)을, **롤아웃** = "기억하기엔 너무 큰 부분"(나머지 깊은 게임)을 담당. 이 분업이 MCTS를 "무작위 시뮬레이션"보다 "무작위 시뮬레이션 + **선택적 기억**"으로 만들며, 15.1절의 "완전탐색 없이 표본추출만으로 최적 전략을 찾는다"는 주장의 **정확한 메커니즘**.

수렴을 추적: 이기는 수의 점유율과 승률

이기는 수(“1개 가져가기”)의 **방문 점유율**과 **추정 승률**을 시뮬레이션 횟수별로 추적(시드 0):

시뮬레이션 횟수	100	200	500	1000	2000	5000	10000
1개 수 방문 점유율	76%	86%	94%	96%	98%	99%	99.5%
1개 수 추정 승률	0.84	0.90	0.95	0.97	0.98	0.99	1.00

- (a) 점유율: 100회 76% $\rightarrow$ 1만 회 **99.5%** 수렴
- (b) “3개 수”는 방문 20∼26회, 승률 0.15∼0.27에서 **안정**
- (a)의 점선 $2/3$ = “무작위 상대” 기준 참 승률 — MCTS 추정치가 **이를 뛰어넘는** 이유가 “트리가 기억하는” 정보 때문



이기는 수의 방문 점유율(좌), 3개 수의 점유율·승률(우).

“1만 회면 충분한가?” 이 게임(남은 깊이 5 이하) 에서는 그렇다. **남은 게임이 짧을수록 수렴은 빠르다** — 바둑처럼 수백 수 남으면 같은 횟수로 같은 정확도는 기대할 수 없으니 AlphaGo는 한 수당 수 초∼수 분의 컴퓨트를 배정.

실습 1: 님(돌 5개)에서 최적 전략 찾기

이 게임의 **수학적 필승 전략**은 잘 알려져 있다 —

상대에게 남기는 돌 수가 **4의 배수**가 되도록 만든다
(돌 5개 ⇒ **1개** 가져가서 4개를 남겨야 최적)

MCTS가 이 규칙을 **전혀 모른 채로도** 스스로 찾아내는지, 4단계를 그대로 구현해 확인:

```
root = mcts_nim(stones=5, iters=3000, seed=0)
for move, child in sorted(root.children.items()):
    print(f"  {move개} 가져가기: 승률={child.wins/child.visits:.3f}, 방문={child.visits}/3000")
```

노트북에서 실행 검증(시드 0∼4로 5회 반복해도 결과 **일관**):

```
개
  1 가져가기: 승률=0.987, 방문=2954/3000개
  2 가져가기: 승률=0.269, 방문=26/3000개
  3 가져가기: 승률=0.150, 방문=20/3000
```

결과: 규칙을 몰라도 98.5%로 필승 수를

- MCTS는 “1개를 가져가서 상대에게 4개를 남기는” 수를 3000번 중 **2954번(98.5%)** 골랐고, 그 수의 승률도 **98.7%**로 가장 높게 — **정확히** 수학적 필승 전략과 일치
- 게임의 규칙 자체(4의 배수 전략)를 MCTS는 **전혀 몰랐는데도**, 오직 “시뮬레이션을 많이 해보고 이긴 비율이 높은 수를 고른다”는 것만으로 최적 전략을 스스로 찾음

- 나머지 두 수(2개, 3개)의 방문이 **0으로 떨어지지 않고 20∼26회**씩 남는 것도 흥미 — UCB 항 덕분에 “덜 시도해본 수”에는 계속 **소량의 탐험 예산**이 배정(Week 2.2의 탐험-활용 트레이드오프가 게임 트리에서도 그대로 작동)

실전에서의 중요성

이 “0으로 안 떨어진다”는 점은 **MCTS의 출력이 “확률적 추천”**이라는 뜻 — 탐색 예산이 부족하면 충분히 확인되지 않은 수에도 방문이 남으므로, **중요한 국면(끝물)에서는 추가 시뮬레이션으로 확신을 높이는** 게 표준적.

실습 2: 더 큰 국면(돌 14개)

4의 배수 규칙은 국면이 커져도 동일해야 한다 — 돌 14개라면 $14 - 2 = 12$를 남겨야 하므로 **“2개 가져가기”가 최적**. 5000회 시뮬레이션(seed 0):

```
개
  1 가져가기: 승률=0.488, 방문=498/5000개
  2 가져가기: 승률=0.830, 방문=4198/5000개
  3 가져가기: 승률=0.441, 방문=304/5000
```

- 방문 점유율: 필승 수(“2개”)에 **84% 집중** — 4의 배수 규칙을 **국면 크기와 무관하게** 찾음
- 그런데 추정 승률이 5개 국면의 98.7%에 비해 **83.0%로 낮다** — “**국면이 클수록 같은 횟수에서 수렴이 느리다**”가 중요한 교훈

“트리 + 롤아웃 합성” 관점으로

“2개”를 둔 뒤 남은 게임이 돌 12개로 **길다** — 트리가 “기억할 수 있는” 짧은 범위의 강제 구조(14개에서 2개를 둔 직후 1∼2수)가 전체 게임에 차지하는 비중이 작아지고, 나머지는 **더 길고 더 노이즈가 큰** 무작위 롤아웃으로 채워지기 때문. 실제로 5만 회까지 돌리면 **97.8%**까지 올라감 — 천장이 낮은 게 아니라 **도달에 시간이 걸리는** 것.

기준선 자체도 내려간다 + 탐색 상수 c

한 겹 더 — "**무작위 상대를 상대로 한 참 승률**"이라는 기준선 자체가 **국면이 클수록 내려간다**:

- 5개: 이기는 수("1개")의 무작위 참 승률 $2/3 \approx$ **67**%
- 14개: 이기는 수("2개")는 $1 - P(12) \approx$ **50.4**% ($P(12) \approx 0.496$ — **거의 동전 던지기**)
- MCTS 추정치(83%)는 이 무작위 참승률(50%)을 **훨씬 뛰어넘지만**, 완전탐색 기준 상한(100%)에는 아직 못 미친 어딘가

실전 바둑에서는 이 두 극단("무작위 참승률"과 "최선 상한")이 **거의 0과 1**로 벌어져, 무작위 롤아웃 하나당 담기는 신호가 **극도로 희박** — AlphaGo가 신경망 롤아웃으로 갈 이유(아래 절)의 극한 버전.

탐색 상수 c를 바꿔도(2000회, seed 0) 이기는 수는 항상 1위지만, 탐험 예산의 분배가 c에 따라 달라진다:

c	1개 수: 승률/방문	2개 수	3개 수
0.8	0.992 / 1983	0.300 / 10	0.143 / 7
1.4	0.982 / 1956	0.280 / 25	0.158 / 19
2.5	0.954 / 1879	0.275 / 69	0.154 / 52

c를 키우면 "덜 시도해본 수"에 배정되는 예산이 커져, 확실히 나쁜 수에도 **69**∼**70회**나 방문이 배분. **c = "확률적으로 보이는 이득"과 "미래 정보 가치"를 어떻게 트레이드오프하느냐의 다이얼** — Week 2.2에서 UCB의 c를 데이터 스케일에 맞춰 튜닝했던 것과 같은 논리. (실제 게임은 $c \approx 1.4$를 기본값으로 두고 검증으로 조절.)

실습 3: 틱택토의 첫 수 (15.1과 연결)

15.1절에서 틱택토를 **완전탐색**(549,946개 국면)으로 풀어 "최선이면 무승부"라는 **정확한** 답을 구했다. 이번엔 MCTS로 — 빈판에서 5000회 시뮬레이션($c = 1.4$, 시드 0/1/2) 후 첫 수 후보 9개:

위치	승률 / 방문 점유율(세 시드 범위)
가운데(4)	**0.72∼0.76** / **29∼41%** (1위)
네 귀퉁이	0.62∼0.68 / 각 9∼13%
나머지(가운데 끼인 변)	0.54∼0.63 / 나머지

- 세 시드 모두 **가운데**가 방문 점유율과 승률 **둘 다에서 1위**

같은 눈으로 읽기

완전탐색의 "무승부" = **양쪽이 모두 최선**일 때의 결과, MCTS의 승률 = **상대가 무작위 플레이**를 할 때의 실전 승률. 틱택토엔 "필승" 수가 없으므로 MCTS의 순위는 "**바보 상대를 상대로** 얼마나 자주 이기고 (상대가 실수할수록 이득이 크고), 얼마나 적게 지는가"를 반영 — 그 관점에서도 "가운데 → 귀퉁이 → 변"은 정석 순서와 일치. **두 방법의 출력을 서로 다른 질문의 답**(정확한 가치 vs 표본 기반 추천)으로 읽는 연습 = 계획 알고리즘을 쓸 때의 기본 태도.

버그 투어: 실제로 발붙여 본 네 가지 실수

MCTS 구현이 "에러 없이 돌면서" 결과가 이상할 때, 원인은 대부분 아래 네 가지 중 하나 (모두 **증상까지 재현**):

1. **선택 루프가 부분 확장 노드에서 멈추지 않는다** — 정지 조건을 `while node.children:`으로 쓰면, 루트 자식 중 **첫 번째로 열린 하나**에 모든 방문이 집중(재현: 3000회 중 한 자식 3000, 나머지 0). UCT는 사실상 작동하지 않고, 어떤 수가 살아남을지는 "우연히 먼저 확장된 수"가 정함. **증상: 결과가 너무 한쪽으로 쏠리고, 시드만 바꿔도 쏠리는 수가 바뀜**
2. **역전파 관점 반전** — 승을 "차례인 플레이어"(`cur.player`)에게 더하면, UCT는 **상대에게 유리한 수**를 "좋은 수"로 착각 — 재현: 방문 **1072/1037/891**으로 거의 균등(1/3씩), 이기는 수를 전혀 구별 못 함, 승률 0.01∼0.03으로 의미 없는 값. **증상: "방문이 고르게 퍼져있다"는 탐험 과대가 아니라 "신호가 완전히 사라진" 경우**
3. **승을 아예 안 더하고 방문만 더** — Q가 항상 0이면 활용 항이 사라져 탐험 항만으로 동작, 방문은 **정확히 1/3씩**(재현: 1000/1000/1000) 균등 — "UCB가 균등 분배한다"는 Week 2.2의 성질이 그대로 재현
4. **터미널 상태에서 확장 시도** — `IndexError`로 크래시(재현: 돌 0개 노드에서 `nim_moves` 빈 리스트). 에러 메시지는 명확하지만 "왜 리프에서 확장을 했지?" 유형

버그 투어의 교훈

네 버그의 공통점:

마지막 하나를 빼면 **에러가 안 나고 "결과가 조금 이상하다"로만** 나타난다 — **계획 알고리즘 디버깅에서 가장 사악한 형태**

- 그래서 이 절의 실습은 "**답을 수학적으로 알 수 있는 게임**(님)" 위에서 한다는 데 의도가 있다: "**방문 분포가 알려진 정답에 집중하는가?**" 자체가 **테스트**
- **답을 알 수 없는 게임(바둑)**에서 디버깅할 때는, 이름을 바꿔가며 위 네 가지 변형을 **직접 만들어가며** "증상과 결과의 대응"을 체득하는 게 표준적 방법

AlphaGo/AlphaZero: MCTS + 신경망

- 순수 MCTS의 시뮬레이션 = **무작위**로 끝까지 — 님처럼 승패가 몇 수 안에 명확해지는 게임에서는 충분히 정확한 신호
- **바둑처럼 한 수의 영향이 아주 나중에** 드러나는 게임에서는, 무작위 롤아웃 하나하나가 사실상 **노이즈**에 가까워 부정확

AlphaGo/AlphaZero의 핵심 개선

무작위 롤아웃을 **신경망**으로 대체:

- **정책(policy)**: "이 상태에서 각 수가 얼마나 유망한가" (Week 10.1의 π_θ와 같은 역할)
- **가치(value)**: "이 상태가 최종적으로 얼마나 유리한가" (Week 3.3의 $V(s)$와 같은 역할) — 다른 헤드 또는 같은 신경망의 다른 출력

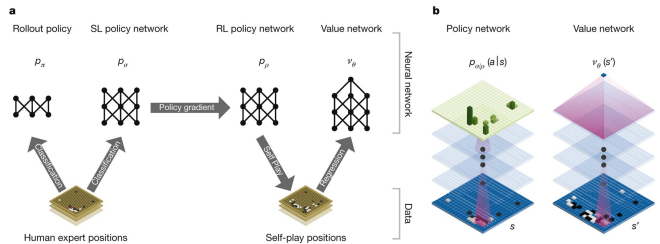
MCTS는 이 추정치를 **길잡이** 삼아 탐색을 훨씬 더 효율적으로 좁혀가고, MCTS로 찾아낸 **더 정확한 수**를 다시 신경망 학습의 **정답(지도학습 목표)**으로 사용 — **탐색(MCTS)과 학습(신경망)이 서로를 개선하는 루프.**

PUCT: 신경망의 직관이 탐색 방향을 정하다

이 결합에서 선택 규칙도 UCT에서 **PUCT**(Policy UCT)로 발전:

$$\text{PUCT}(s,a) \;=\; \frac{Q(s,a)}{N(s)+N(s,a)} \;+\; c_{\text{puct}}\,P_\theta(s,a)\,\frac{\sqrt{N(s)}}{1+N(s,a)}$$

- **차이 = 탐험 항**: UCT의 $c\sqrt{\ln N(s)/N(s,a)}$ 는 시도 안 된 수에 **균등하게** 보너스, PUCT는 **정책 신경망의 사전확률** $P_\theta(s,a)$로 **가중** — "**신경망의 직관**"이 탐색의 방향을 정하고, MCTS가 그 직관을 **검증·수정하는 통계**로 되돌려주는 식
- 활용 항도 $Q/(N+N(a))$로 **부드럽게** 정교해짐



정책망·가치망 학습 파이프라인(좌)과 두 신경망의 합성곱

학습 루프 한 바퀴:

1. **자기 자신과의 대국(self-play)**으로 경험 생성
2. MCTS로 각 국면 탐색한 **방문 분포**로 **정책망** 지도학습
3. 게임 **최종 승패**로 **가치망** 지도학습
4. 더 나은 망으로 **다시** self-play

왜 바둑에 이런 무기가 필요했는가: 숫자

19×19 바둑판에서 단 **81수만** 두어도 가능한 수순은 $19^{81} \approx 10^{104}$개:

- 한 판(양쪽 합쳐 280∼300수, 한쪽 약 80수)은 그 트리의 **약 80개 리프**만을 탐색
- **100억 판**(10^{10})을 끝없이 두어도 도달하는 국면은 약 $8 \times 10^{11} \approx 10^{12}$개 — 전체 국면 수 10^{170}의 10^{-158} 부분

완전탐색이 성립하지 않는 규모

"유망한 가지만 **표본추출로 파고든다**"는 MCTS의 원칙(15.1절 틱택토 실습의 격차의 **극한**)이 사실상 **유일한** 길 — 그리고 그 표본추출의 방향을 **신경망이 가중해준 PUCT + self-play 루프**가 **이세돌 승부**를 만들어냄.

이 절 주제(목표 기반 RL과 계획)를 더 깊이 다루는 자료: Stanford CS234.

자주 하는 실수: 롤아웃 정책과 시뮬레이션 횟수

- **① 롤아웃 정책을 너무 정교하게 만들려는 것** — "무작위 롤아웃은 부정확하니 정교한 규칙을 넣자"는 생각에 복잡한 로직을 넣으면, 시뮬레이션 **한 번의 계산 비용**이 커져 같은 시간 안에 **훨씬 적은 횟수**만 돎 — 위 님 실습처럼 **"단순하지만 많이" 도는 쪽이** 나을 때가 많음
  - AlphaZero가 신경망으로 롤아웃을 대체한 것도 "더 똑똑하게"가 아니라 "**한 번의 평가로 승패 시뮬레이션 수백 번을 대체**할 만큼 신뢰도 높은 신호를 준다"는 **효율성 계산**
- **② UCT 탐험 상수 c를 문제와 무관하게 고정**하는 것 — $c = 1.4$는 님 게임에는 잘 맞았지만, 승/패 대신 **연속적인 점수**를 다루는 문제에서는 값의 **스케일**이 다르므로 c를 다시 튜닝해야 (Week 2.2에서 UCB의 c를 데이터 스케일에 맞춰 조정해야 했던 것과 같은 이유)

자주 묻는 질문(15.2)

- **Q: 틱택토(완전탐색)와 님(MCTS)은 왜 다른 방법?** — 틱택토는 549,946개로 완전탐색이 가능해 minimax로 “100% 정확한” 답. 님은 **사실** 완전탐색이 가능하지만, **일부러** MCTS로 풀어 “완전탐색 없이 표본추출만으로도 최적 전략을 찾아낼 수 있다”를 보여주기 위한 예제 — 실전에서 MCTS를 쓰는 이유는 바둑처럼 완전탐색이 **아예 불가능한** 문제이기 때문
- **Q: MCTS는 지도학습? 강화학습?** — **엄밀히 둘 다 아님** — **계획(planning)** 알고리즘. Week 4 가치반복처럼 모델(규칙)이 있다는 전제하에 “지금 무엇을 할지”를 계산, Week 9–11처럼 경사하강으로 파라미터를 갱신하는 게 **아님**. AlphaZero가 강한 이유 = 정확히 이 계획(MCTS)과 학습(신경망 지도학습)을 **루프로 엮었기 때문**
- **Q: “가장 많이 방문된 자식” vs “승률 최대 자식”?** — 표준은 **방문 최대(most visits)**. 방문이 적을 때 승률 추정치는 **노이져스**(20번 중 3승 vs 100번 중 30승은 승률 같아도 신뢰도 다름), 방문 횟수는 UCT가 “여기를 더 봐야겠다”고 판단한 **양** 자체가 견고. 시뮬레이션이 충분히 많으면 대개 같은 수를 가리키고, 예산이 끝났을 때 승률 격차가 확실히 큰 **예외 상황**에서만 승률 최대 변형

자주 묻는 질문(15.2, 이어서)

- **Q: “승률 98.7%”는 무작위 상대와의 실전 승률(67%)과 같은 의미인가?** — **아니** — 이 구분이 “트리와 롤아웃의 역할 분담” 절의 핵심. 98.7% = **트리가 “상대 대응에 대한 내 필승 응답”을 통계에 저장한 뒤**의 **합성 추정치**, 순수 무작위 승률($2/3$)을 **뛰어넘음**. 시뮬레이션을 늘리면 님처럼 **끝이 짧은** 게임에서는 100%(최선-최선 기준 참 승률)에, 바둑처럼 **긴** 게임에서는 “무작위 참승률”과 “최선 상한” **사이에** 수렴 — 그 위치가 바로 **신경망 롤아웃이 개입**하는 곳
- **Q: 시뮬레이션은 몇 번이면 충분한가?** — **닫힌 형태의 공식이 없음** — (1) **수렴 곡선이 평탄해지나**(이기는 수의 방문 점유율이 고정값에 다가서는가), (2) **수의 순위**가 시뮬레이션 추가에 대해 **변하지 않는가**로 실측해 판단. 예산 배분 경험칙 = “**중요한 수 (결정적 국면)에, 쓸 수 있는 만큼**” — AlphaGo는 한 수당 CPU 수 초∼수 분, 남은 깊이가 짧은 **끝물**은 수렴이 훨씬 빨라 같은 횟수로 더 정확한 추정

연습문제(15.2)

1. **(수치 계산)** 부모 방문 $N(s) = 111$, 자식 $A : (Q, N) = (10, 100)$, $B : (5, 10)$, $C : (0, 1)$, $c = 1.4$일 때 세 자식의 UCT 값 계산 후 어떤 수가 선택되는지 판정. (힌트: $\ln 111 \approx 4.71$. "승률 0인 C"가 **다시** 선택될 수 있을까?)
2. **(개념 서술)** 선택 단계의 정지 조건 `len(node.children) == len(nim_moves(...))`가 없으면(`while node.children:`으로 대체) 어떤 증상이 나서 **왜** 그런지 "확장의 보장"과 연결지어. (힌트: 버그 투어 1의 재현 값 인용.)
3. **(코딩)** `mcts_nim`에서 c를 0으로 바꿔 3000회 실행 — 방문 분포가 어떻게 되고, **어느 자식**에 집중되는지 예측(힌트: 어떤 수를 **먼저** 확장하는지는 `rng` 순서에 의존) 후 확인. "탐험 항 $c\sqrt{\ln N/N}$의 역할"에 대해 무엇을 말하는가?
4. **(개념 서술)** 님(5)에서 이기는 수의 MCTS 추정 승률(0.987)이 "무작위 상대 참 승률"(2/3)보다 큰 이유를, "트리가 기억하는 부분"과 "롤아웃이 표본추출하는 부분"의 **분업**으로. 이 갭은 어떤 게임에서 커지고, 어떤 게임에서 작으며, 왜? (힌트: 남은 게임의 **깊이**.)

15.3 멀티에이전트 RL 개관 (선택)

에이전트가 하나가 아니라면

- 지금까지는 **에이전트가 하나뿐인** 환경을 다뤘다
- **멀티에이전트 강화학습(Multi-Agent RL, MARL)**: 여러 에이전트가 **같은 환경에서 동시에** 행동 (협력: 팀 스포츠·물류 로봇 협업, 경쟁: 바둑·대전 게임 AI)
- 근본적 차이: 환경이 더 이상 **고정된 규칙만** 따르지 않음 — 다른 에이전트도 학습하며 계속 전략을 바꿈

움직이는 표적(non-stationary target)

"**내 입장에서 본 환경**"이 학습 도중에 **계속 움직이는 표적** — Week 3.1의 "환경은 고정된 $P(s'|s,a)$를 따른다"는 **마르코프 성질의 전제** 자체가, 다른 에이전트를 환경의 일부로 보는 순간 흔들림.

- 단일 에이전트 MDP: 기대리턴은 **자기 정책만의 함수** $J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_t \gamma^t r_t\right]$
- 에이전트가 둘이면: $J_1(\pi_1, \pi_2)$, $J_2(\pi_1, \pi_2)$ — **자기 정책과 나머지 모든 에이전트의 정책**의 함수

“계산이 큰 문제”가 아니라 “성질이 다른 문제”

- π_2가 **정해져 있으면** 에이전트 1의 최적 정책 $\pi_1^* = \arg\max_{\pi_1} J_1(\pi_1, \pi_2)$도 **존재**
- 그런데 π_2가 **바뀔 때마다** “최적 정책”이라는 **것 자체가 바뀜**
- Week 6 Q-learning이 수렴하던 표적 $Q^*(s, a)$는 **고정**되어 있었는데, 멀티에이전트에서는 내가 수렴해야 할 **“정답”이 상대의 학습과 함께 계속 이동**

이번 세 실습이 보여줄 것

학습 알고리즘은 **모두 똑같은 단순한 Q-learning / fictitious play**이고, 결과가 **완전히** 달라지는 것은 오직 **보상 구조(게임의 규칙)** 때문.

죄수의 딜레마: MARL을 배우는 2×2 게임

경찰이 두 용의자를 **따로** 심문 — 각자 **C(침묵, coop)**와 **D(자백, defect)** 중 하나. 보상 = 클수록 좋게 변환:

	에이전트 2: C (침묵)	에이전트 2: D (자백)
에이전트 1: C (침묵)	(3, 3)	(0, 5)
에이전트 1: D (자백)	(5, 0)	(1, 1)

역사: 1950년경 Merrill Flood·Melvin Dresher가 RAND에서 실험을 설계하며 만들었고, "죄수의 딜레마"라는 이름은 Albert W. Tucker가 붙임. 1928년 John von Neumann가 2인 제로섬 게임의 "최적 전략" 존재를 증명한 것이 게임이론 의 시작이라면, 죄수의 딜레마는 **제로섬이 아닌** 게임이 얼마나 다른지를 보여준 첫 예제.

- ① **지배전략이 D다** — 상대가 C든 D든 내가 D가 더 큼(C면 5>3, D면 1>0): "상대의 전략을 알아야 고르는" 게 아니라 **"무조건 D"**가 개인 합리적
- ② **그런데 (D,D)는 (C,C)보다 둘 다에게 나쁨** — 둘 다 개인적으로 합리적이면 (1,1)에, 둘 다 C이면 (3,3)을 나눌 수 있었음

MARL의 핵심 긴장 — "**각자 합리적인 학습의 결과가 공동으로 최적이지 않을 수 있다**"를 최소 크기(2×2)로 응축.

내쉬 균형: 이해의 첫 걸음

내쉬 균형(Nash equilibrium, 1950, John Nash)

남의 전략이 주어진 상태에서, 어느 한쪽도 단독으로 전략을 바꿨을 때 이득을 보지 못하는 전략 프로파일.

- **(D,D) 확인**: 상대가 D인 상태에서 내가 C로 바꾸면 보상 $1 \to 0$으로 **줄어듦** — 한쪽의 단독 이탈로 이득이 없으므로 **(D,D)는 내쉬 균형**
- **(C,C)는?**: 상대가 C인 상태에서 D로 바꾸면 $3 \to 5$로 **늘어남** — 이탈 유인이 있으므로 **내쉬 균형이 아님**

- 정식 게임이론(존재 증명, 혼합전략 확장, 보상 분배, "내쉬 균형을 찾는 문제는 **PPAD-완전**일 만큼 계산상 어려울 수 있다" 등)은 이 학기 **범위 밖**
- 오늘은 내쉬 균형을 "**학습 시뮬레이션의 결과를 읽는 데 쓰는 언어**" 정도로만 — 세 실습의 결과를 내쉬 균형의 눈으로 보면, "알고리즘이 뭘 잘못한 것"처럼 보이는 결과가 **게임 구조의 필연**인 것이 보임

실습 1: 죄수의 딜레마 자기학습

두 에이전트가 같은 보상 행렬을 놓고, **둘 다 Week 6의 ε-greedy Q-learning으로 배우며 서로를 상대로 플레이(self-play)** — 알고리즘은 각자가 자기 Q값을 갱신할 뿐(상대도 학습 중이라는 사실은 **어디에도 명시되지 않음**):

```
import numpy as np
# 행에이전트=1 행동, 열에이전트=2 행동 (0=C 침묵, 1=D 자백)
PD = np.array([[3.0, 0.0],    # (C,C)=3, (C,D)=0
               [5.0, 1.0]])  # (D,C)=5, (D,D)=1

def eps_greedy(Q, eps, rng):
    if rng.random() < eps:
        return int(rng.integers(len(Q)))
    return int(np.argmax(Q))

def self_play(games, payoff, alpha=0.5, eps_start=1.0, eps_end=0.02,
              tau=500.0, seed=0):
    rng = np.random.default_rng(seed)
    Q1, Q2 = np.zeros(2), np.zeros(2)
    rewards1 = []
    for g in range(games):
        eps = eps_end + (eps_start - eps_end) * np.exp(-g / tau)  # 탐험감소
        a1 = eps_greedy(Q1, eps, rng)
        a2 = eps_greedy(Q2, eps, rng)
```

결과: 둘 다 “완벽하게” 배우면 → 상호 배신

실행 결과(seed=0):

- 초기 500경기 평균 보상 ≈ **1.96** — C와 D가 뒤섞인 탐색기
- 경기 **2,500**경기를 지난 뒤부터 (D,D)가 경기의 **98% 이상**
- 마지막 1,000경기 평균 보상 = **1.012** — **이론값 1.0과 일치**

핵심

둘 다 “완벽하게” Q-learning을 잘한 결과는 **상호 배신 (D,D)**. 학습 알고리즘이 **아무것도 잘못하지 않았다** — 각자의 관점에서 “상대가 D를 하니까 나도 D가 최선”이라는 추론이 매 스텝마다 정확했다. **“각자 합리적으로 학습한” 결과가 공동으로 (C,C)보다 나쁜 내쉬 균형으로 수렴** — 단일 에이전트에서는 “알고리즘이 정확히 수렴했다”면 성공이었지만(Week 6–7), 멀티에이전트에서는 **수렴하는 대상 자체가 문제의 구조(보상 행렬)에 의해 정해진다**.

대조 실험: 상대가 항상 C면?

- 상대를 **"항상 C(침묵)를 고르는 고정 에이전트"**로 바꾸고 동일하게 학습시키면:
- 마지막 1,000경기 평균 보상 = **4.976**(이론값 5.0)로, 에이전트는 당연히 **D(자백)**로 수렴 — "상대가 배신하니까 나도 배신한다"가 아니라 "**상대가 침묵하니까 내가 자백해서 5를 챙긴다**"는 방향

결정하는 것은 알고리즘이 아니라 게임

내쉬 균형 (D,D)로 수렴하든, 상대를 **착취**하는 D로 수렴하든 — **결정하는 것은 알고리즘이 아니라 게임**.

실습 2: 조율 게임 — 균형을 "고르는" 문제

교차로에서 두 차가 동시에 다다름: **둘 다 좌회전(L)** 또는 **둘 다 우회전(R)**이면 안전하게 지나감 (2,2), 방향이 다르면 **충돌** (0,0):

$$R = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{(행=에이전트1, 열=에이전트2, L=0, R=1)}$$

죄수의 딜레마와 다른 점:

- ① **지배전략이 없다** — "무조건 L"도 "무조건 R"도 아님, 상대와 **맞추는 것**이 중요
- ② **내쉬 균형이 둘** — (L,L)도 (R,R)도 내쉬 균형(한쪽만 바꾸면 2에서 0으로) — "정답"이 둘이고 둘 다 "합법"

동일한 `self_play`를 그대로 돌려보면(게임만 COORD로 교체):

- 마지막 1,000경기 중 **97.8%가 같은 방향** 선택, 그중 에이전트 1이 L을 고른 비율 **98.6%** — 이 실행에서는 **(L,L) 균형으로 잠금(lock-in)**
- **시드를 바꾸면 (R,R)에 잠기는 실행도** 나온다는 것

교훈: 경로 의존(path dependence)

협력 게임에서 학습의 문제는

"최해를 **찾느냐**"가 아니라 **"어느 균형으로 수렴하느냐"** — **경로 의존(path dependence)**.
초기 탐색의 **잡음**(어느 쪽에 먼저 방문 횟수가 쌓였느냐)이 최종 균형을 결정.

- 현실에서도 이 구조는 **흔하다**:
  - **VHS와 Betamax**의 비디오테이프 포맷 전쟁(1980년대) — 기술적 우열이 아니라 "어떤 쪽이 **먼저 잠금**되는지"를 가른 조율 게임
  - 오늘날의 **통신 표준**(Wi-Fi, USB-C) 경쟁도 같은 구조 — 표준화 기구가 존재하는 이유도 "학습으로 균형이 **잘못 잠기면 안 된다**"는 인식

실습 3: 바위-가위-보 — 순수 전략이 없는 게임

바위(0) > **가위(1)**, **가위(1)** > **보(2)**, **보(2)** > **바위(0)**인 제로섬 게임(이기면 $+1$, 지면 -1, 비기면 0):

- **순수 전략의 내쉬 균형이 없다** — 상대의 순수 전략(예: 항상 바위)을 알면 항상 보를 내면 되므로, "상대가 바위를 낼 것"이라는 예측은 상대가 알아채고 전략을 바꿈으로써 **무너지는 순환**
- 이 게임의 내쉬 균형은 **혼합 전략(mixed strategy)**으로만 존재 — 각 행동을 **정확히 1/3씩** 독립적으로 섞는 전략

확인하는 수학

상대가 $(1/3, 1/3, 1/3)$이면 내가 어떤 순수 전략을 내도 기대 보상 $= \frac{1}{3}(+1) + \frac{1}{3}(0) + \frac{1}{3}(-1) = 0$로 **동일** — 어떤 순수 전략도 이탈 이득을 주지 않음(제로섬이므로 상대도 0).

- Q-learning만으로는 이 균형을 **"행동"으로 재현하기 어려움** — ε-greedy는 결국 한 행동에 argmax가 기울어져 **잠기려는 성향**(실제로 특정 행동에 잠기거나, 잠긴 채로 상대에게 **착취**당함)

fictitious play: 빈도 추측 + best response

그래서 제로섬 게임에서는 더 단순한 알고리즘이 쓰인다 — **fictitious play**(가상 플레이, 1951년 John Robinson): 각 에이전트가 **상대가 지금까지 낸 행동의 빈도**를 추정하고, 그 빈도에 대해 **best response**(기대 보상을 최대화하는 행동)을 한다. "상대가 최근 바위를 많이 냈다" ⇒ "다음에는 가위" — **예측과 반응만 반복할 뿐 Q값이 없다**:

```
def fictitious_play(games, payoff, seed=0, eps=0.1):
    """제로섬: 각에이전트는상대의행동경험빈도에       best response.
    eps 확률로균등랜덤을섞어순수의      BR 고착을피한다 ."""
    rng = np.random.default_rng(seed)
    n = payoff.shape[0]
    freq1 = np.ones(n) / n    # 에이전트1 행동의누적빈도
    freq2 = np.ones(n) / n    # 에이전트2 행동의누적빈도
    for t in range(games):
        br1 = int(np.argmax(payoff @ freq2))      # 1: 상대빈도에  BR
        br2 = int(np.argmin(payoff.T @ freq1))    # 2: zero-sum →의 1 기대보상최소화
        a1 = br1 if rng.random() < (1 - eps) else int(rng.integers(n))
        a2 = br2 if rng.random() < (1 - eps) else int(rng.integers(n))
        freq1 = (t * freq1 + np.eye(n)[a1]) / (t + 1)
        freq2 = (t * freq2 + np.eye(n)[a2]) / (t + 1)
    return freq1, freq2

RPS = np.array([[0, 1, -1],      # 바위(0)가위>(1), 가위(1)보>(2), 보(2)바위>(0)
                [-1, 0, 1],
```

결과: 둘 다 혼합 균형 (1/3, 1/3, 1/3)으로

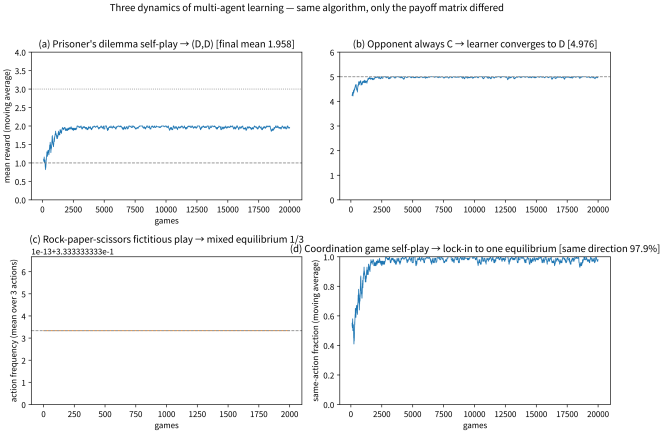
실행 결과 — 에이전트 1 최종 빈도 $[0.326,\ 0.331,\ 0.344]$, 에이전트 2 $[0.342,\ 0.338,\ 0.321]$:

둘 다 혼합 균형 $(1/3, 1/3, 1/3)$으로 수렴

"상대의 패턴을 읽고 반격을 준비한다"는 **국소적 규칙**을 반복했을 뿐인데, 그 **반복의 한계**가 게임의 혼합 균형과 일치 — 제로섬 게임에서 fictitious play가 내쉬 균형으로 수렴한다는 것은 **Robinson(1951)의 고전 결과**.

- 참고: $\varepsilon = 0.1$의 **작은 랜덤성**은 "순수 BR만 쓰면 **대칭이 깨진 순수 전략쌍에 고착**될 수 있다"는 실습에서 발견한 실제 동작을 피하기 위한 것 (확인 문제 3에서 이 역할을 생각)

세 가지 동역학: 알고리즘은 같고, 보상만 달랐다



(a) PD self-play: 초기 혼합(평균 ≈2) → (D,D) 수렴, 최종 1.012(이론 1.0). (b) 상대가 항상 C인 고정 에이전트: 학습한 에이전트가 D로 수렴, 최종 4.976(이론 5.0). (c) RPS fictitious play: 둘 다 1/3 혼합 균형. (d) 조율 게임: (L,L) 잠금(lock-in), 최종

- **동역학의 "형"은 보상 구조가 결정**:
  - (a) 상호 배신 = 게임의 내쉬 균형이 (D,D)
  - (d) 잠금 = 균형이 **둘**이라 초기 조건이 선택지를 결정
  - (c) 순수 균형 **자체가 없어** 혼합 균형이 나옴

협력 vs 경쟁, 그리고 self-play

MARL 문제는 크게 **두 극단**으로 나뉨:

완전 협력

공통 보상을 공유하는 여러 에이전트 (예: 물류창고의 로봇 여러 대가 함께 짐을 옮기는 문제).

완전 경쟁

제로섬 게임 — 한쪽이 얻으면 다른 쪽이 정확히 그만큼 잃음(예: 바둑, 체스). 실전 문제는 이 **둘 사이 어딘가**에 많음(자율주행차: 충돌 회피에서는 협력, 도로 공간을 두고는 경쟁 — Week 14의 단일 에이전트 로봇과 대비).

self-play: 세 실습과 AlphaZero를 잇는 관념

AlphaZero가 **스스로와 대국하며** 강해진 것도 멀티에이전트 학습의 **특수 형태** — 상대가 매번 "지금 막 학습한 **나 자신의 복사본**"이라, 상대 실력이 내 실력과 함께 계속 높아지는 **자연스러운 커리큘럼**.

self-play가 효과적인 이유

- **(1) 난이도가 학습자와 동행** — “너무 쉬워서 배울 것이 없어지거나” “너무 어려워서 신호가 없어지는” 극단이 **없음**
- **(2) “나보다 약간 강한 상대”를 이기려는 학습**은 **정밀하게 차이만 교정**하는 신호를 만들어 학습 효율이 **좋음** (15.2절 MCTS가 신경망보다 정확한 수를 찾아줬던 것과 같은 이치)

- Week 5의 몬테카를로 학습처럼 “무작위 행동으로 시작해 경험을 쌓아야 하는” 학습에서는 **초기 신호의 품질**이 병목 — self-play는 “**초기부터 좋은 상대를 대결시킨**” 버전

다만

이 “좋은 상대”를 사람이 만들어준 것(인간 게임 데이터, AlphaGo(2016)의 경우)이 아니라 **학습 과정 자체가 만들어내는** 것이 **AlphaZero(2017)의 특징**.

MCTS와 미니맥스: 경쟁 게임의 연결고리

- 15.2의 MCTS는 사실 “상대방도 **최선을 다한다**”고 가정하는 **2인 제로섬 게임**에서는 **미니맥스(minimax)** 탐색의 **확률적 근사**로 볼 수 있음 — 15.1의 틱택토 완전탐색(minimax)과 15.2의 MCTS가 풀려던 문제(**다음 수 정하기**)는 사실 **같은 문제**, 다만 상태 공간이 커서 완전탐색이 안 될 때 **표본추출로 근사**한 것이 MCTS
- 이것을 “협력이 아니라 **경쟁**”이라는 멀티에이전트의 한 형태로 보면, ML1 Week 15.3의 **GAN**에서 본 **min-max 게임**(생성자와 판별자가 서로 다른 목표로 상호작용)과도 **구조적으로 닮음** — 둘 다 “한쪽의 이득이 다른 쪽의 손해인 두 참여자”라는 틀을 최적화 문제로 표현

GAN 훈련의 불안정성 $\Leftrightarrow$ RPS

GAN의 교대 최적화(ML1 15.3)는, 위 바위-가위-보에서 “순수 전략의 내쉬 균형이 없어 **균형을 향해 진동하며** 수렴한다”는 사실의 **최적화 버전** — **경쟁 게임의 학습 동역학이 본질적으로 “균형으로의 수렴”보다 “균형 근처의 진동”에 가깝다**는 인식이 GAN과 MARL 양쪽에서 같은 안정화 아이디어를 유도(ML1 15.3의 GAN 학습 스케줄링, MARL에서 **상대의 이전 체크포인트와 대국하기** 등).

자주 하는 실수: “단일 에이전트 RL을 N번 돌리면 된다”

가장 흔한 실수 — “각 에이전트에 Week 6 Q-learning을 각각 돌려주면 되지 않나”(단일 에이전트 관점을 그대로 N개로 복제). 무너지는 점 세 가지:

1. **상대를 “환경”의 일부로 보면 마르코프 성질이 깨진다** — “내 관점의 환경” $P(s'|s,a)$는 상대의 정책 π_2에 의존, 상대가 학습 중이면 **전이확률이 학습 도중에 바뀜**. Week 6–7의 Q-learning 수렴 증명은 “**고정된 MDP**”를 전제로 했으니, 그 보증은 **상대가 고정된 정책**(예: 실습 1의 “항상 C” 에이전트)을 사용할 때만 유효 — 상대가 학습하면 수렴이 느려지거나, **상대의 학습 궤적에 과적합된** “그 상대를 겨냥한” 정책이 나올 수 있음
2. **“둘 다 잘 배우면 좋은 공동 정책이 나온다”는 직관이 틀릴 수 있다** — 실습 1이 정확히 이를 보임: 둘 다 “완벽하게” 배우면 (D,D)라는 **공동으로 열위한** 결과에 수렴. 학습의 품질(개인의 리턴 최대화)과 **공동의 최적**은 **일반적으로 다른 목표** — 협력 게임에서 “공동 보상”을 정의하는 **보상 설계**가 이 둘을 일치시키거나(실패)하지 않음
3. **“수렴”이 내쉬 균형으로의 수렴이라는 보장을 받지 않는다** — 실습 2처럼 **균형이 여러 개**이면 학습 궤적(시드, 초기화)이 어느 균형으로 갈지 결정, 실습 3처럼 **순수 균형이 없으면** 혼합 전략(또는 그 근처의 진동)만이 가능한 목표. “학습이 끝났다”는 신호(보상 이동평균의 평탄화)가 “**좋은 균형**에 도착했다”는 신호와 같은 것은 **아님**

자주 묻는 질문(15.3)

- **Q: MARL의 실제 알고리즘(QMIX, MAPPO 등)은 무엇을 하나?** — 위 세 실습의 “단순 Q-learning + self-play”에서 크게 **세 방향**으로 발전: (1) **중앙 집중 학습 + 분산 실행** — 훈련 때는 모든 에이전트 상태를 한데 모아 한 가치함수를 학습(Centralized Training), 실행할 때는 각자가 자기 관측만으로 행동(Decentralized Execution) = “움직이는 표적” 문제를 **“훈련 중에는 표적을 고정시킨다”**로 우회 (2) **보상(가치) 분해** — 협력 게임에서 “공동 가치”를 각 에이전트의 가치합으로 분해(QMIX, 2018) (3) **경쟁형 self-play + MCTS** — AlphaZero 계열. 이번 학기 목적 = “**왜 단일 에이전트 관점이 깨지는가**(non-stationarity)” + “**보상 구조가 동역학을 정하는가**” 이해
- **Q: 에이전트끼리 소통하면 상황이 바뀌나?** — **네, 근본적으로** — 소통은 행동에 “메시지” 채널을 더하는 것으로, 협력 게임에서 “내가 무엇을 할지”를 공유하면 조율 게임의 **경로 의존성을 대폭 감소**(두 차가 방향을 합의하면 충돌은 원칙적으로 없어짐). 다만 소통 자체가 “누가 무엇을 말할지”를 학습하는 **또 하나의 멀티에이전트 문제**를 만들므로 별도 연구 주제

자주 묻는 질문(15.3, 이어서)

- **Q: 세 게임은 “동시 선택”인데, MCTS 게임 (바둑)은 “순차 선택”이 아닌가?** — **맞다, 두 축**이 있다: (1) **협력 vs 경쟁**(보상 구조) = 이번 절의 주제, (2) **순차 vs 동시**(선택의 시간 구조) — 바둑/체스는 번갈아 놓으므로 “상대의 다음 수”를 MCTS처럼 **탐색**으로 고려 가능, 조율 게임처럼 **동시 선택**에서는 상대의 선택을 “읽을” 시간이 없으므로 탐색이 아니라 **전략(확률 분포)의 학습**이 핵심. MCTS = (경쟁, 순차) 각의 방법, 이번 절의 Q-learning/fictitious play = 주로 (협력/경쟁, 동시) 각 — 바둑 AI(AlphaZero)가 두 관념을 **동시에** 쓰는 게임(경쟁 + 순차 + 상대가 학습)의 **한계점**에 위치 — 이 장 전체가 하나의 이야기 로 이어지는 이유
- **Q: GAN min-max와 경쟁 게임 학습이 실제로 같은 수학인가?** — **구조적으로 비슷하고 실무적 교훈을 공유** — 둘 다 “상대의 파라미터도 동시에 움직이는” min-max를 경사하강으로 푸니, **순차 경사하강**(한쪽 고정)은 “고정된 상대” 전제의 근사, 실제로 양쪽이 함께 움직이므로 **진동(cycling)**이 흔함. GAN의 “교대 최적화의 함정”(생성기 좋아지면 판별기가 다시 바뀌고)과 **모드 붕괴**가 이 구조적 불균형의 표현, MARL에서는 “내 정책이 좋아졌는데 상대가 대응 전략을 배우면 다시 나빠진다” 로 — 둘 다 “상대의 학습 속도를 내 것보다 **늦추거나** (self-play에서 상대의 **이전 체크포인트**와 대국), **소프트하게** 만들어서” 진동을 줄이는 **같은 계열의 안정화 기법**

15.3 핵심 정리

위 세 실습은 모두 “**동일한 학습 규칙이, 보상 구조에 따라 완전히 다른 동역학** (상호 배신, 균형 잠금, 혼합 전략 수렴)을 만들었다”를 보여줌

MARL에서 “학습 알고리즘”보다 “게임의 구조(보상, 정보, 선택의 순서)”가 결과를 결정.

- 단일 에이전트 RL이 “**알고리즘 설계의 학문**”에 가까웠다면, 멀티에이전트 RL은 “**게임 설계의 학문**”에 가까워짐 — 자율주행, 로봇 협업, 경매 AI 같은 실전 문제는 거의 항상 “게임의 구조를 **제대로 설계하는 것**”(보상을 어떻게 나눌지, 정보를 얼마나 공유할지)이 알고리즘 선택보다 **먼저 결정문제가 됨**

그리고 MCTS는 “모델이 있다면(규칙을 안다면), 실제로 행동하기 전에 미리 수를 읽어볼 수 있다”는 아이디어를 **극한**까지 밀어붙인 방법 — Week 7.3 Dyna-Q가 학습된 모델로 **값**을 개선했다면, MCTS는 완벽한 모델로 **지금의 결정 자체**를 개선. AlphaZero가 보여줬듯, 이 탐색과 신경망 학습을 결합하면 **순수 탐색도, 순수 신경망도** 각각 해내지 못한 성능에 도달.

15.4 World Models: 학습된 모델로 상상하며 계획하기

known model에서 learned model로, 그리고 “상상”으로

15.1절에서 모델기반 RL을 **known model**(체스·바둑처럼 규칙이 완전히 주어짐)과 **learned model**(모델 자체를 데이터로 배워야 하는 대부분의 실전 경우)로 나눴다.

World Model = “learned model”을 극단까지 밀어붙인 아이디어

환경의 **다이내믹스** ($s_{t+1} = f(s_t, a_t)$) **자체를 신경망으로 통째로 학습**한 뒤, 그 신경망 **안에서** 에이전트를 훈련.

- 진짜 환경과의 상호작용은 **학습된 모델을 만들 때만** 쓰고, 정책을 다듬는 **대부분의 학습**은 그 모델이 “**상상**”하는 **가짜 경험 (imagined rollout)** 위에서 일어남
- **왜 가능**: 실제 환경 스텝보다 **신경망 순전파가 훨씬 싸** — 같은 계산 예산으로 **훨씬 많은 “경험”**을 만들어낼 수 있음

대표 연구 두 갈래

World Models (Ha & Schmidhuber, 2018)

- 이 아이디어를 **분명하게 보여준 초기 작업**
- (1) **VAE**(ML1 Week 15)로 화면 이미지를 **압축된 잠재 벡터** z로 생성
- (2) **RNN(MDN-RNN)**으로 그 잠재 공간 위에서 "다음 z가 어떻게 될지"를 예측하는 **다이내믹스 모델** 학습
- (3) 이 **압축된 잠재 공간 안에서만** 컨트롤러(정책) 학습 — **진짜 게임 화면은 한 번도 다시 보지 않고도** 정책이 완성

Dreamer (Hafner et al., 2020)

- 이 아이디어를 **현대 딥러닝 스케일**로 발전
- 잠재 공간에서의 **상상된 궤적을 통째로 역전파**해 정책과 가치함수를 **동시에** 학습 (**actor-critic을 상상 공간에서**)
- **실제 로봇 제어** 같은 **연속 행동 공간·고차원 관측**(카메라 이미지)에서도 **데이터 효율적**으로 동작을 보임

왜 이게 중요한가: 데이터 효율성

- **model-free**(DQN, PPO)는 좋은 정책을 얻기까지 **수백만∼수억 스텝**의 실제 환경 상호작용이 필요할 수 있음
- **로봇처럼 "실제 스텝"이 비싸거나 위험한** 환경에서는 이게 **큰 걸림돌**

World Model 계열의 전략

"환경과는 **적게** 상호작용하고, 그 대신 **학습된 모델 안에서 많이 상상한다**" — 이 문제를 **정면으로** 공략.

- Week 14의 시뮬레이터(MuJoCo, Isaac Sim) = "**물리 법칙을 사람이 정확히 코딩해서 준** 모델"
- World Model = "**그 모델 자체도 데이터로부터 배우자**"는 **한 걸음 더 나아간 발상**

실습: CartPole을 “상상 속에서만” 조종

이 절의 노트북은 위 아이디어를 **CartPole**으로 아주 작게 재현:

1. **무작위 행동**으로 모은 (s_t, a_t, s_{t+1}) **9,297개**로 작은 MLP **다이내믹스 모델**을 80 epoch 학습
2. 1스텝 예측 오차: 각도 기준 θ RMSE **0.067 rad (0 epoch)** $\rightarrow$ **0.0013 rad (79 epoch)**

그다음부터는 실제 환경(`env.step`)을 **한 번도 부르지 않고**, 이 모델 **안에서만** 정책을 굴려 봄 (“상상 롤아웃”, `imagine`):

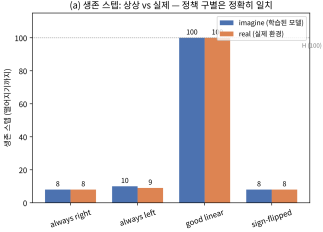
	고정 행동 1	고정 행동 2	손으로 짠 선형 제어	부호 뒤집은 고장난 제어
생존 스텝(상상)	8	10	100	8
생존 스텝(실제)	8	10	100	8

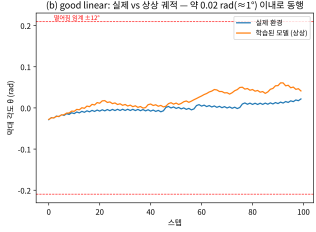
네 자리 모두 정확히 일치

학습된 모델이 실제 환경의 동역학을 충분히 잘 복제했다는 뜻.

한계: 1스텝 오차가 쌓이면

- 동시에 확인되는 것: 스텝을 계속 이어 붙이면 **1스텝 오차가 쌓여**(오차의 **지수적 증폭**) **20스텝쯤부터** 상상 궤적과 실제 궤적의 각도가 **서서히 벌어짐**





CartPole의 learned dynamics: 1스텝 예측 오차의 감소와, 이어 붙인 상상 궤적과 실제 궤적이 20스텝쯤부터 벌어지는 모습.

이 지점이 곧 한계

World Model이 "**완벽한 시뮬레이터**"가 아니라 **유한한 정확도의 근사**라는 한계를 **숫자로** 직접 보는 지점.

15주차 정리

1. **15.1 — 두 축의 지도** — 모델기반 RL = **모델의 출처**(known/learned) × **용도**(학습/ 계획). 틱택토 549,946개 vs 바둑 2×10^{170}: "모델을 안다" $\neq$ "계산할 수 있다". 가치반복은 **한 반복당 한 칸**, 깊이 시뮬레이션은 **압축**. Dyna-Q: 계획 10스텝으로 334$\rightarrow$52 에피소드(약 6.4배), 30으로 35(한계 효과 감소) — **목적지는 같고 도로의 길이만** 달라. "모델이 있으니 학습 불필요"는 **양쪽(known/learned) 모두**에서 성립하지 않음.

2. **15.2 — MCTS = UCB + 끝까지 해보기** — **선택**(UCB의 형태 UCT) · **확장** · **시뮬레이션**(무작위 롤아웃) · **역전파** (승패 통계 전달). 님(5)에서 4의 배수 전략을 **규칙을 몰라도** 98.5%로 찾음. 추정 승률 98.7% $>$ 무작위 참 승률 67% = **트리(기억) + 롤아웃(표본추출)의 합성**. 탐색 상수 c = 탐험 예산의 **다이얼**. 버그 투어: 에러 없이 "조금 이상한 결과"가 가장 사악함. AlphaZero = **PUCT**(정책망 사전확률로 가중) + self-play 루프; 100억 판으로 10^{170}의 10^{-158}.

3. **15.3 — 게임 설계의 학문(선택)** — **비정상성**: 상대가 학습하면 수렴해야 할 "정답" 자체가 움직임. 알고리즘(단순 Q-learning/fictitious play)은 **모두 같고** 결과가 달라지는 것은 오직 **보상 구조** — 죄수의 딜레마(상호 배신, 최종 1.012), 조율 게임((L,L) **잠금**, 경로 의존, VHS/Betamax), 바위-가위-보(혼합 균형 1/3). "단일 에이전트를 N번 돌린다"는 실수는 **마르코프 성질 파괴·공동 최적 $\neq$ 개인 최적· 균형 수렴 무보증** 세 곳에서 무너짐. self-play가 AlphaZero를 잇는 관념.

4. **15.4 — 학습된 모델 안에서 상상** — **World Models**(VAE + MDN-RNN, 잠재 공간 안에서만 컨트롤러)와 **Dreamer**(상상 궤적을 통째로 역전파)가 learned model을 신경망 스케일로 밀어붙임. CartPole 실습: 상상 생존 스텝이 **실제와 네 자리 모두 일치**(8/10/100/8), 20스텝쯤부터 오차가 쌓여